TetrioCoach: An Adaptive Tetris AI Coaching System Based on Large-Scale Replay Data Analysis

ChangHo Lee
Independent Researcher

Abstract

This study presents TetrioCoach, an adaptive AI coaching system that leverages replay data from the competitive puzzle game TETR.IO. Whereas prior research on Tetris AI has concentrated on maximizing the in-game performance of autonomous bots, the proposed system establishes a prescriptive coaching pipeline that automatically classifies a human player's weaknesses and generates skill-adapted training feedback. A hybrid classifier—trained by labeling 61,935 real matches (7,716,524 piece placements) from the top 500 ranked players via K-Means clustering and learning these labels with Gradient Boosting—achieved a Macro F1-score of 0.960. Through a cross-recommendation matrix of 16 build patterns and 11 rank tiers, the system produces strategic advice differentiated by player skill level. The complete 4,087-line system operates independently without any external LLM dependency, and articulates a new research direction: the transition from descriptive statistics to prescriptive feedback.

Keywords: game AI, player modeling, replay analysis, unsupervised clustering, gradient boosting, skill calibration, prescriptive feedback, domain-knowledge-based recommendation, esports analytics, human-AI interaction

AI Disclosure Statement

Anthropic's large language model Claude was used as an assistive tool during system design, code implementation, literature search, and structuring of this manuscript. Claude assisted with code drafting, academic literature retrieval, and suggesting the outline structure of the paper; however, all research-design decisions, experimental design, interpretation of results, and final judgment of academic claims were made by the author, ChangHo Lee. Because Claude cannot bear research accountability or legal responsibility, it is not included in the author list. This disclosure is provided transparently in accordance with the AI-use guidelines of major venues such as IEEE, ACM, and Nature.

I. Introduction

1.1 Background and Motivation

Since its creation by Alexey Pajitnov in 1984, the competitive puzzle game Tetris has remained, for four decades, one of the most widely played video games worldwide. The online versus platform TETR.IO, in particular, recorded more than 9.5 million registered users and over one billion cumulative games as of 2026, illustrating how the competitive dimension of Tetris has grown into a significant segment of the esports ecosystem.

Nevertheless, coaching systems that systematically support player improvement remain underexplored, both academically and commercially. Approaches that rely on human coaches scale poorly, while the recently emerging LLM-based coaching carries inherent limitations such as hallucination, API cost, and non-deterministic output.

1.2 Research Objectives and Questions

Prior to any clinical validation of coaching efficacy, the objective of this study is to demonstrate the algorithmic validity and computational efficiency (<100 ms) of a comprehensive pipeline that converts 7.7M fragmented placement-level play records into natural-language feedback. In other words, this paper constitutes a proposal of a

system architecture and data-abstraction framework rather than a clinical study; a quantitative user study of the efficacy of the prescriptive feedback is explicitly designated as future work (Section 5.2).

Within this scope, this study seeks to answer the following three research questions:

RQ1. Can player weakness types be automatically classified from large-scale replay data?

RQ2. Can feedback grounded in the classified weaknesses be differentiated across player skill levels?

RQ3. Does structuring domain-specific knowledge (build patterns, tier benchmarks) improve feedback quality?

1.3 Contributions

The central contribution (hook) of this work is as follows:

> *"Whereas prior Tetris AI research (Dellacherie, 2003; Thiery & Scherrer, 2009; Gabillon et al., 2013) focused on how well a bot plays, this study addresses a fundamentally different question: how effectively a human player can be taught."*

- Descriptive-to-prescriptive shift: beyond descriptive statistics, we build a pipeline of ML-based weakness classification → tier benchmark comparison → tailored training prescription.
- Multi-level data abstraction: 7.7M placement-level records are transformed into natural-language coaching through six abstraction stages.
- Structured domain knowledge: the first cross-recommendation matrix of 16 build patterns × 11 rank tiers.

II. Theoretical Background

2.1 Academic Lineage of Tetris AI

Tetris has been proven NP-complete (Breukelaar et al., 2004), and the number of possible states on a 10×20 board reaches 2^{200}, rendering an exact MDP solution intractable (Bodoia & Puranik, 2012). Heuristic-based approaches have therefore dominated the field.

Dellacherie (2003) proposed a linear evaluation function composed of six features—landing height, row and column transitions, holes, wells, and eroded cells—achieving an average of 660,000 cleared lines; Thiery & Scherrer (2009) subsequently validated and improved this line of work using the Cross-Entropy method. Gabillon et al. (NeurIPS, 2013) surpassed these results with approximate dynamic programming, and Cold Clear (MinusKelvin, 2019) evolved the weights of more than 20 features via a genetic algorithm, becoming the de facto standard for modern Tetris bots. Erkalkan (2026) reported a +61.79% performance gain over fixed weights through GA-based dynamic weight adjustment.

2.2 AI Coaching in Esports

Research applying game AI to player coaching has grown rapidly in recent years. Google DeepMind's TacticAI (2024) applied ML to football tactical analysis, and Jing et al. (2025) analyzed twelve years of professional Counter-Strike performance using SHAP-based interpretable ML. However, an integrated framework that automatically classifies weaknesses from replay data and combines them with domain-specific knowledge to generate prescriptive feedback has not yet been systematically established in the literature.

2.3 Machine-Learning-Based Player Profiling

Gradient Boosting (Friedman, 2001) sequentially constructs an ensemble of weak learners to maximize classification and regression performance, and is known to perform strongly on tabular data. In this study, it is used to classify a player's principal weakness type (speed, attack, tspin, finesse, defense, balanced) from a 10-dimensional feature vector extracted from real game data.

III. Methodology

3.1 System Architecture

TetrioCoach is designed as a four-layer architecture: (L1) Data Acquisition—TETR.IO API, local .ttrm files, the Kaggle dataset, and the build-pattern database; (L2) Statistical Analysis—the aggregation engine, board simulator, play-style analyzer, and build detector; (L3) AI Intelligence—the ML weakness classifier, rule-based evaluator, and heuristic board evaluator; (L4) Feedback Generation—the coaching report, training roadmap, build advisor, and matchup advice.

[Fig. 1: TetrioCoach Multi-Layer Architecture — insertion point]

3.2 Data Collection and Preprocessing

Data are collected through three channels. (1) The public TETR.IO API (ch.tetr.io/api) provides leaderboard and match-record queries, yielding summary statistics such as APM, PPS, VS, and garbage sent/received. (2) Direct parsing of local .ttrm files extracts detailed statistics—T-Spin types (TSS/TSD/TST/Mini), line clears, finesse, and placement counts—from the replay.results.stats path. (3) The public Kaggle dataset (n3koasakura, 2024) supplies 7,716,524 placement-level records from 61,935 matches of the top 500 players; each placement includes the board state (playfield), T-Spin type, combo, B2B, garbage, and Glicko-2 rating.

3.2.1 All-Tier Benchmark Collection

Because placement-level detail (T-Spin, finesse, etc.) exists only in the Kaggle dataset (top 500 players), the ML style classifier is trained on elite data. To let the evaluation/feedback layer assess lower-tier players against the distribution of their own tier rather than against absolute thresholds, we randomly sampled 100 players from each of 11 tiers (C, B, B+, A, A+, S, S+, SS, U, X, X+; the X+ tier contains only 80 players in total) from the public TETR.IO leaderboard API, collecting the APM/PPS/VS/TR distributions of 1,080 players in all (Table 3). This separation is a central methodological decision of this work: style archetypes are learned most cleanly from low-noise elite data, whereas skill-level calibration is performed using the empirical distributions of every tier.

[Table 3] Empirical all-tier benchmark (1,080 players; 100 per tier, 80 for X+)

Tier	TR (mean)	APM	PPS	VS
X+	24,317	169.8	3.27	339.8
X	23,209	130.8	2.82	266.3
U	22,229	107.8	2.49	223.0
SS	19,928	79.2	2.04	163.3
S+	17,879	57.5	1.79	122.1
S	16,223	46.4	1.63	98.8
A+	13,248	33.8	1.37	72.3
A	11,466	27.2	1.24	59.2
B+	8,145	20.9	1.11	45.3
B	6,435	18.9	1.04	40.5
C	2,539	13.0	0.87	27.0

APM/PPS/VS increase monotonically with tier, and tier estimation is performed by nearest-centroid matching (normalized Euclidean distance) against these 11 tier means. In a self-consistency check, the mean profile of every one of the 11 tiers was classified back to its own tier.

3.3 Statistical Aggregation Engine

More than 30 metrics are computed from the collected round data: win rate (match/round), mean APM/PPS/VS, counts and ratios by T-Spin type, line-clear distribution (Single/Double/Triple/Quad), finesse fault ratio, garbage pressure ratio, maximum combo/B2B/spike, and APM/PPS trends. Differences in data availability between the online API and local .ttrm files are managed via a has_detail flag, so that missing data are handled explicitly in both the UI and the AI analysis.

3.4 ML Weakness Classifier

To avoid circular reasoning, the ML classifier adopts a two-stage approach. (Stage 1) K-Means clustering (k=8) derives natural clusters from the 10-dimensional feature vectors of 20,000 games. The ten features are: (1) PPS, (2) estimated APM (attack/piece × 60), (3) T-Spin rate (%), (4) Quad rate (%), (5) lines per piece, (6) maximum combo, (7) maximum B2B, (8) maximum board height, (9) defense ratio (garbage cleared/received), and (10) rating_norm (TR/1000). Weakness labels are then mapped from each cluster's profile (the z-scores of these features relative to the global mean). In this way, labels are derived from the structure of the data itself rather than from hard-coded rules over the input variables. (Stage 2) A GradientBoostingClassifier (n_estimators=200, max_depth=6) is trained on the cluster labels to predict the weaknesses of new players.

At inference time, a hybrid prediction combines the ML probability (weight 0.4) with a rule-based score (weight 0.6). This compensates for the scale mismatch between the training data (top 500 players) and general players. Class imbalance is addressed with noise-injection oversampling (min_ratio=0.10), and final performance is reported with the Macro-averaged F1-score rather than plain accuracy.

Rationale for the two-stage pipeline (K-Means → Gradient Boosting): This structure performs surrogate learning of the geometric decision boundary of K-Means with a supervised model, so a high F1-score is mathematically expected.

The benefits of nonetheless adopting the two-stage structure are: (1) whereas K-Means assigns a discrete label to new data based solely on the distance to the nearest centroid, Gradient Boosting provides a probability distribution (predict_proba) that enables soft judgments on borderline cases; (2) when ML probabilities are weighted and fused with rule-based scores during hybrid inference, a supervised model that yields continuous probabilities permits more flexible fusion than discrete cluster assignment; and (3) inference proceeds immediately with the pre-trained classifier without re-running K-Means, securing the computational efficiency suited to the real-time response (<100 ms) of the GUI application.

[Table 1] Performance of the ML Weakness Classifier (K-Means labeling + oversampling)

Class	Precision	Recall	F1	Support
attack	0.97	0.97	0.97	1,293
defense	0.96	0.97	0.96	1,106
speed	0.96	0.96	0.96	1,017
tspin	0.95	0.93	0.94	584
Macro avg	0.96	0.96	0.96	4,000

3.5 Build-Pattern Knowledge Base

A structured database of 16 build patterns—collected from the Hard Drop Wiki and FOUR.lol (TKI, DT Cannon, PCO, Hachispin, Albatross, MKO, C-Spin, STSD, LST Stacking, Imperial Cross, Fractal, 4-wide, 6-3 Stacking, Downstacking, etc.)—was constructed. Each build records its category (opener/midgame/strategy/defense), required bag count, attack type, output lines, prerequisites, coverage, difficulty, applicable tier, and strength/weakness/counter strategy; from these, a matrix of optimal build recommendations across 11 tiers (C–X+) was assembled.

To ensure the objectivity of the build-tier mapping, the applicable tier for each build was determined by cross-referencing (1) the per-build difficulty and prerequisites specified in the Hard Drop Wiki and FOUR.lol, and (2) community statistics and open-source guides on the TR ranges in which each build is used in practice (e.g., 4-wide is effective at intermediate ranges but is countered at higher ranges, whereas continuous T-Spin builds such as Albatross/Fractal are mainly used in the X+ range). The mapping rules thus rest on publicly available domain knowledge and community consensus rather than the author's arbitrary judgment. Quantitative validation of this mapping by an expert panel remains future work.

3.6 Feedback Generation Pipeline

Feedback generation proceeds as follows: (1) statistical aggregation via compute_aggregates_v2() → (2) construction of a PlayerProfile via build_profile_from_agg() → (3) generation of a WeaknessReport list via evaluate_profile() (6 categories, 4 severity levels, priority scores) → (4) ML weakness prediction via predict_weakness() → (5) generation of a coaching report (8 sections) + training roadmap (tier-specific 20-minute routine) + matchup advice (counters to opponent builds). The entire process completes locally within 100 ms without any external API call.

IV. Results

4.1 ML Model Performance

After K-Means-based labeling and oversampling, the Gradient Boosting classifier achieved an Accuracy of 0.962, a Macro F1-score of 0.960, and a 5-fold CV Macro F1 of 0.956 (±0.003) on the test set (n=4,000). To avoid the well-known problem whereby plain accuracy is inflated by majority classes under class imbalance, this study adopts the Macro F1-score as the primary evaluation metric. Oversampling balanced the support across the four classes (attack, defense, speed, tspin) to a range of 584–1,293 (improved from the previously extreme imbalance of 12–12,908), so that Accuracy (0.962) and Macro F1 (0.960) converge to within 0.002. This indicates that classification performance is uniformly secured across all classes, and the small gap between the two metrics attests to the success of the balancing.

Feature-importance analysis showed that lines_per_piece (28.6%) was the most important variable, followed by rating_norm (21.7%), max_btb (15.9%), and tspin_rate (6.4%). The dominance of tspin_rate (73.9%) in the previous version was attributable to the circular reasoning of rule-based labeling; under unsupervised clustering, line efficiency and overall rating emerged as the key variables for weakness discrimination. This implies that the model discriminates weaknesses through a multi-dimensional combination of features rather than a single metric.

A note on the rating_norm variable. In this study, rating_norm is a scale-normalized variable defined as the Kaggle dataset's TR (Tetra Rating) divided by 1000 (mean ≈ 24.9 for the top 500 players). Because the training data are concentrated in the single X+ band, its standard deviation is small (0.095); the model uses subtle inter-cluster rating differences as an auxiliary signal during training, but at inference time the new player's TR is unknown and is therefore fixed to the training mean (24.9). Consequently, the 21.7% importance of rating_norm reflects a contribution within the training distribution; in actual inference it acts as a population-level anchor rather than a discriminative variable distinguishing individual players. Replacing rating_norm with measured values once all-tier data are obtained is a future improvement that would restore its discriminative power.

[Fig. 2: Feature Importance Bar Chart — insertion point]

4.2 Comparison with Prior Approaches

[Table 2] Comparison with prior Tetris AI approaches

Study	Objective	Method	Output
Dellacherie (2003)	Optimal play	6-feature heuristic	Placement
Gabillon et al. (2013)	Optimal play	ADP value function	Placement
Cold Clear (2019)	Optimal play	GA weight evolution	Placement
Erkalkan (2026)	Optimal play	GA dynamic weights	Placement
TetrioCoach (this work)	Human coaching	ML classification + rules + domain knowledge	Natural-language feedback

4.3 Case Studies

Case 1: Longitudinal Growth Analysis (Subject P, 37 matches over one year)

One year of replay data (37 .ttrm files) from a single player was partitioned into three periods and analyzed: early (2025-06 to 08), mid (2025-12 to 2026-03), and recent (2026-06).

[Table 4] Longitudinal growth trajectory of Subject P (over one year)

Period	Rounds	APM	PPS	Win%	T-Spin %	Fault%	ML pred.
Early (2025-06~08)	31	49.3	1.81	54.8%	4.1%	61.3%	attack (0.48)

Mid (2025-12~2026-03)	41	45.9	1.75	56.1%	3.9%	58.3%	attack (0.52)
Recent (2026-06)	16	55.1	1.92	62.5%	4.3%	55.8%	finesse (0.56)

Over one year, APM rose from 49.3 to 55.1, PPS from 1.81 to 1.92, and the win rate from 54.8% to 62.5%. The finesse fault improved from 61.3% to 55.8% but remained at a critical level. Notably, the hybrid ML prediction ranked "attack" first in the early and mid periods (0.48–0.52) and shifted to "finesse" (0.56) in the recent period (with the secondary weakness also alternating between attack and finesse). This is interpreted as the model capturing a growth pattern in which, as APM improved from 49.3 to 55.1, the attack weakness was relatively resolved while finesse emerged as the new principal bottleneck. The fact that the weakness diagnosis migrates over time within a single subject's longitudinal data demonstrates that the model does not simply converge to a fixed label but responds to changes in the input metrics.

Case 2: A High-Level, Balanced Player (Player A, anonymized)

With PPS 2.36 (fast), APM 54.1, a T-Spin rate of 4.33%, and a finesse fault of 6.8% (excellent), all core metrics are balanced and accuracy is particularly strong. The hybrid ML model predicted "balanced" as the top label (confidence 0.58), and the rule-based evaluator likewise drove the finesse weakness score nearly to zero. Based on an estimated S tier, the system recommended TKI, DT Cannon, Hachispin, and STSD, and generated maintenance-oriented feedback to "sustain the current level while targeting the next tier." This shows that the system correctly identifies a balanced player with no salient weakness.

Case 3: A Slow, Low-T-Spin Player (Player C, anonymized)

With PPS 1.56 (slow), APM 50.9, a T-Spin rate of 1.55% (low), and a finesse fault of 17.9% (sound), the hybrid ML model predicted "speed" as the top label (confidence 0.58), and the rule-based evaluator likewise detected the low PPS as the principal weakness. Accuracy (fault 17.9%) is relatively sound, but the player exhibits a pattern of insufficient speed and T-Spin utilization; the system, based on an estimated A tier, recommended TKI/DT Cannon/STSD and placed "three 40L sprints as a speed-sense warm-up" first in the 20-minute routine, generating a prescription that prioritizes speed improvement above all.

Case 4: Multi-Player Comparative Analysis (N=38)

Sample selection criterion. From the author's local match archive (37 .ttrm files), all 37 distinct opponents with at least 6 rounds (one full match) of data were selected after nickname deduplication, and Subject P (recent period) was added, for a total of N=38. The sample thus follows an objective criterion—"every opponent in the archive with sufficient data"—rather than hand-picking. The sample spans a wide range: PPS 1.38–2.37, APM 31.8–95.7, and finesse fault 6.8–70.0%.

Distribution of ML top-1 weaknesses (N=38). attack 22 (57.9%), balanced 8 (21.1%), speed 3 (7.9%), finesse 3 (7.9%), defense 2 (5.3%). All five weakness classes appear in the sample, confirming at the N=38 scale that the model does not collapse to a single label but differentiates according to differences in the input profile. The prevalence of attack (57.9%) reflects the fact that the archive consists mainly of S–S+ band opponents, so the shared characteristic of that skill band (insufficient attack-conversion efficiency) is reflected.

[Table 5] Representative subset spanning all five weakness classes (anonymized P1–P5 + Subject P)

Metric	P1	P2	P3	P4	P5	Subj.P
PPS	2.36	2.33	1.56	2.05	1.77	1.92
APM	54.1	31.8	50.9	62.7	50.2	55.1
T-Spin%	4.33	0.0	1.55	4.39	1.03	4.3
Fault%	6.8	43.1	17.9	42.5	42.1	55.8
Tier	S+	S	S	S+	S+	S+

ML top-1	bal.	att.	spd.	fin.	def.	fin.
Conf.	0.58	0.58	0.58	0.41	0.35	0.56

The six representative players each exemplify one of the five weakness classes: P1 (fault 6.8%, PPS 2.36) is balanced; P2 (no attack vehicle at 0% T-Spin) is attack; P3 (slow at PPS 1.56) is speed; P4 (fault 42.5%) is finesse; and P5 (T-Spin 1.03%, defense-leaning) is defense. Even within the same tier band (S–S+), the diagnosis differentiates clearly according to differences in the input profile.

Error analysis of the weakness distribution. The majority share of attack (57.9%) at N=38 is not a model bias but stems from the skill-band homogeneity of the sample (mostly S–S+). Within the same band, a subset with uniformly high finesse faults tends to converge to finesse; however, when players with heterogeneous accuracy, speed, and T-Spin utilization are included (Table 5), the predictions diverge normally into the five classes balanced/attack/speed/finesse/defense. A structural characteristic nonetheless remains: because the rule-based weight (0.6) exceeds that of the ML model (0.4), the rule score prevails in borderline cases; securing all-tier training data would allow the ML weight to be raised to strengthen data-driven classification.

Qualitative contrast of final prescriptions. Between players with different ML top-1 labels, the downstream prescriptions diverge clearly. For P1 (judged "balanced"), the first step of the 20-minute routine was "10 repetitions of the TKI opener pattern" (mastering attack builds); for P3 (judged "speed"), "three 40L sprints as a speed-sense warm-up"; and for P2 (judged "attack"), opener-build study was placed first. This demonstrates that the combination of ML label → tier estimation → build matrix → weakness priority generates qualitatively distinct training prescriptions per player.

Case 5: Subject P's Growth Against the All-Tier Benchmark (Table 3)

From the empirical all-tier benchmark collected in Section 3.2.1 (Table 3, 1,080 players), we extracted the A–SS range spanned by Subject P's growth trajectory and compared the longitudinal change in position. The benchmark figures use the same authoritative data as Table 3 (removing the noise of the earlier partial S–U sample).

[Table 6] Position of Subject P against the all-tier benchmark (A–SS, excerpt of Table 3)

Group/Tier	TR	APM	PPS	VS	n
A tier avg	11,466	27.2	1.24	59.2	100
A+ tier avg	13,248	33.8	1.37	72.3	100
S tier avg	16,223	46.4	1.63	98.8	100
S+ tier avg	17,879	57.5	1.79	122.1	100
SS tier avg	19,928	79.2	2.04	163.3	100
Subject P early	-	49.3	1.81	108.7	31R
Subject P mid	-	45.9	1.75	94.1	41R
Subject P recent	-	55.1	1.92	119.4	16R

Subject P's early metrics (APM 49.3, PPS 1.81) lie between the S tier average (APM 46.4, PPS 1.63) and the S+ average (APM 57.5, PPS 1.79), and the recent metrics (APM 55.1, PPS 1.92) are close to the S+ average (the system's estimated tier is likewise S+). APM falls slightly short of the S+ average (57.5) while PPS exceeds it, so the all-tier benchmark objectively supports the system's diagnosis that "attack-conversion efficiency" is the next growth point. The one-year rise from the S level to near-S+ provides indirect corroborating evidence consistent with the improvement direction suggested by TetrioCoach's feedback.

V. Discussion

5.1 Academic Significance

Whereas prior Tetris AI research (Dellacherie, 2003; Gabillon et al., 2013; Cold Clear, 2019) focused on optimal bot play (producing placements), this study makes a paradigmatic contribution by repurposing the same domain for prescriptive coaching of human players. Concretely, the academic significance can be summarized in four points.

(1) Descriptive-to-prescriptive shift: beyond mere descriptive statistics, we present the first comprehensive pipeline that converts 7.7M placement records into natural-language training prescriptions through six abstraction stages.

(2) Weakness classification without circular reasoning: by deriving labels from unsupervised K-Means clusters, the hardcoded-rule circularity is avoided while attaining a Macro F1 of 0.960.

(3) Separation-of-concerns design: style classification uses elite data while skill-level calibration uses the empirical distributions of all 11 tiers (1,080 players), structurally preventing the propagation of single-dataset bias. This is a design instance that explicitly separates the role of each data source in esports player modeling.

(4) Structuring of domain knowledge: by building a 16 build patterns × 11 tiers cross-recommendation matrix grounded in public community knowledge, informal domain expertise is formalized into reproducible prescription rules.

Through these contributions, this study opens up the new research area of replay-based automated coaching and provides a framework generalizable beyond Tetris to any competitive game for which replays exist (Puyo Puyo, fighting/rhythm games, etc.).

Justification of the data design by separation of concerns. The system trains its ML style classifier on the curated placement-level data of the top 500 players. This is a deliberate choice: rather than contaminating the classification boundary with noisy lower-tier data, it learns the geometric decision boundary of the most idealized build efficiency and play styles more precisely. At the same time, skill-level calibration is carried out separately using the empirically collected distributions of all 11 tiers (1,080 players, Table 3), so that lower-tier players are also assessed against the norms of their own tier. By separating style classification (elite data) from level calibration (all-tier data), the design structurally prevents the bias of any single dataset from contaminating both. The fixing of rating_norm at inference time, discussed in Section 4.1, is a consistent corollary of this separation.

5.2 Limitations

• Tier range of the ML training data: because placement-level detail exists only for the Kaggle top 500, the ML style classifier is trained on elite data. The evaluation/tier-calibration layer is complemented with the empirical distributions of all 11 tiers (1,080 players, Section 3.2.1); however, placement-level data (T-Spin, finesse, etc.) for lower tiers are collectable only with a bot account, so retraining the classifier itself on all-tier data remains future work.

• Labeling methodology: although K-Means clustering improves upon the circular reasoning of rule-based labeling, comparison against a gold-standard label set cross-validated by human experts (coaches) is still needed.

• Board simulator: DAS/ARR timing mismatches limit the fidelity of line-clear reconstruction.

• Hybrid prediction structure: because the rule-based weight (60%) exceeds that of the ML model (40%), a tendency to converge to the same label was observed when the analyzed group shares a common weakness (high finesse fault). This can be improved by raising the ML weight once all-tier training data are obtained.

• Absence of a user study: to quantitatively verify the efficacy of the prescriptive feedback, at minimum a small-scale pilot study (2–3 players per tier, tracking TR change) is required but was not conducted here. The case analyses in this paper constitute indirect, post-hoc consistency verification.

5.3 Ethical Considerations

Individual consent was not obtained from the players whose replay data were used in the case analyses of this study. Accordingly, the nicknames of all players, including the author, were anonymized as Subject P, Player A, Player B, and Player C. The Kaggle dataset used to train the ML model (n3koasakura, 2024) is a public dataset collected through TETR.IO's public API and contains no personally identifying information. Any future user study will proceed through IRB (Institutional Review Board) approval.

5.4 Future Work

- Complete a placement-accuracy comparison system via a direct build of Cold Clear 2 (Rust).
- Collect all-tier replays and retrain the model by securing a TETR.IO bot account.
- Conduct an A/B user study tracking TR change between TetrioCoach users and non-users.
- Extend to real-time coaching with an instant-feedback system via in-game board-state capture.
- Generalize the framework to other competitive games such as Puyo Puyo and rhythm games.

VI. Conclusion

In the new research area of Tetris AI coaching, this study designed and implemented a complete pipeline—large-scale replay data analysis → ML-based weakness classification → domain-knowledge-based prescriptive feedback generation—and indirectly verified the system's consistency through case analyses. The 4,087-line system operates independently without any external LLM, and the classifier—trained by labeling 61,935 real matches via K-Means clustering—achieved a Macro F1-score of 0.960. Through a cross-recommendation matrix of 16 build patterns and 11 tiers, the system delivers tailored coaching to players at all levels, from beginner to top tier. This framework can generalize beyond Tetris to automated coaching for replay-based competitive games, and articulates a new paradigm for AI coaching: the transition from descriptive to prescriptive.

References

[1] Algorta, S. & Simsek, O. (2019). The Game of Tetris in Machine Learning. arXiv:1905.01652.

[2] Bodoia, M. & Puranik, A. (2012). Applying Reinforcement Learning to Competitive Tetris. Stanford CS229 Project Report.

[3] Breukelaar, R., Demaine, E.D., Hohenberger, S., Hoogeboom, H.J., Kosters, W.A., & Liben-Nowell, D. (2004). Tetris is Hard, Even to Approximate. International Journal of Computational Geometry & Applications, 14(1-2), 41-68.

[4] Dellacherie, P. (2003). Algorithm for Tetris. Unpublished manuscript.

[5] Erkalkan, E. (2026). Heuristic Optimization of a Tetris Bot Using Genetic Algorithms: An Adaptive Evolutionary Approach. Turkish Journal of Mathematics and Computer Science.

[6] Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. Annals of Statistics, 29(5), 1189-1232.

[7] Gabillon, V., Ghavamzadeh, M., & Scherrer, B. (2013). Approximate Dynamic Programming Finally Performs Well in the Game of Tetris. Advances in Neural Information Processing Systems (NeurIPS), 26, 1754-1762.

[8] Jing, S. et al. (2025). Interpretable Machine Learning with SHAP for Esports Performance Analysis of Professional Counter-Strike Players from 2012 to 2025. International Journal of Sports Science & Coaching.

[9] MinusKelvin. (2019). Cold Clear: Tetris Bot. GitHub. https://github.com/MinusKelvin/cold-clear

[10] n3koasakura. (2024). Tetr.io Top Players Replays (Tetris). Kaggle Dataset. https://www.kaggle.com/datasets/n3koasakura/tetr-io-top-players-replays

[11] Stevens, M. (2016). Playing Tetris with Deep Reinforcement Learning. Stanford CS231N Reports.

[12] Thiery, C. & Scherrer, B. (2009). Improvements on Learning Tetris with Cross Entropy. International Computer Games Association Journal, 32(1), 23-33.

Appendix A: Visual-Material Planning Guide

A.1 Fig. 1 — System Architecture Flowchart

A four-layer structure (Data Acquisition → Statistical Analysis → AI Intelligence → Feedback Generation), with the constituent modules and data flow shown for each layer. To be produced as a high-resolution vector image based on the SVG diagram generated while drafting this outline.

A.2 Fig. 2 — Feature-Importance Analysis

(a) Horizontal bar chart: importance ranking of the 10 features, emphasizing lines_per_piece (28.6%), rating_norm (21.7%), and max_btb (15.9%) in order. Visually contrasted against the previous version's tspin_rate (73.9%) overfitting to convey the improvement toward a multi-dimensional distribution. (b) Confusion matrix (4×4): attack/defense/speed/tspin. (c) ROC curve: multi-class, one-vs-rest.

A.3 Fig. 3 — Feedback Generation Pipeline

Input (.ttrm/API) → parsing → aggregation → [ML classifier + rule evaluator + build DB] → coaching + roadmap + matchup. Visualizing the data transformation and abstraction level at each stage.

A.4 Table 3 — Build-Pattern × Tier Recommendation Matrix

A cross-tabulation of 16 builds (rows) × 11 tiers (columns), with check marks for builds recommended at each tier, annotated with each build's attack type, difficulty, and output lines.

A.5 Fig. 4 — User Dashboard Screenshots

(a) Growth-graph tab: 6 subplots (APM, PPS, VS, garbage, T-Spin/Quad, Finesse). (b) Statistics-summary tab: showing the online/local branch. (c) AI-coaching tab: 8-section feedback text. (d) Training-roadmap tab: the 20-minute routine.